

O instantâneo em produção

Proposta de projeto, não de implementação. O objetivo é decidir **onde o `hig_mesh_snapshot` vive, quem o produz e o que o invalida** antes de encostar nos sítios de consulta — porque varrer sem isso produz código que fica verde e errado, e cinco dos arquivos com mais sítios são solvers que nenhum exemplo alcança.

Tudo abaixo foi medido em `higflow/src` em 20 de setembro de 2026, com o `src_hugo` excluído da contagem (ele não entra na compilação).

Estado em 21 de setembro de 2026 — o projeto saiu de proposta e está no `master`. Tudo o que este documento propõe foi construído, verificado e mesclado (`22b331e .. ee88ecd` , 56 commits); a ramificação `mesh-t8code` foi apagada, porque o trabalho deixou de ser uma ramificação e virou o projeto. As seções abaixo ficam **como foram escritas**, com o raciocínio no tempo em que ele aconteceu — o que mudou depois está marcado em cada seção e reunido na seção 7. Documento de projeto que se reescreve para parecer que sempre soube o resultado perde justamente o que o torna útil: onde o autor errou.

1. O que a medição mudou no plano

A adaptação existe, e reconstrói em vez de mutar

Eu havia afirmado aqui que **não há adaptação de malha em tempo de execução**. Estava errado, e o erro foi de medição: grepei `higflow/examples` , diretório que não existe — os exemplos são `higflow/example2d_*` e `example3d_*` . O `2>/dev/null` engoliu o *No such file or directory* e eu li zero resultados como ausência de adaptação.

Medido de novo, nos 14 exemplos:

<code>higflow/src</code>	0 sítios de <code>hig_refine_uniform</code> / <code>hig_split</code>
<code>example2d_ElectroOsmotic</code>	2 sítios
<code>example2d_DynamicMeshAdapt</code>	2 sítios

O que a leitura de cada um mostra é que **a premissa de projeto sobrevive, por um motivo diferente do que eu tinha escrito**:

- `example2d_ElectroOsmotic` , sítio 1: refina um `hig_clone` da raiz, usado só para escrever VTK e destruído em seguida. A malha viva não é tocada.
- `example2d_ElectroOsmotic` , sítio 2: está em `adapt_mesh_and_update_solver` , que refina a malha viva — e cuja **única chamada está comentada** (linha 364).
- `example2d_DynamicMeshAdapt` : o caminho vivo é `higflow_rebuild_with_amr` , que constrói uma árvore adaptada nova, chama `lb_calc_partition` e **reconstrói** o solver sobre ela.

Ou seja: a adaptação em tempo de execução **existe**, e ela funciona reconstruindo o domínio, não mutando o que está vivo. Nenhum caminho alcançável refina uma árvore que já pertence a um `sim_domain` .

Para o instantâneo isso é melhor do que a premissa errada que eu tinha: como a adaptação passa por `psd_create` / `psd_synced_mapper` outra vez, **o instantâneo é reproduzido sozinho**. Continua não havendo política de invalidação a desenhar.

Mas existe uma mina, e ela é o motivo de o detector não ser supérfluo. Duas funções de refino *no lugar* estão escritas e dormentes:

<code>higflow_refine_tree_inplace</code>	definida, nunca chamada
<code>adapt_mesh_and_update_solver</code>	definida, chamada comentada

Nenhuma das duas passa pela API do domínio, então a guarda de `sd_add_higtreen` não as vê. Se alguém as acordar, o instantâneo fica velho **em silêncio** e os laços migrados passam a ler a célula errada. Quem detecta esse caso é o `sd_snapshot_is_current`, e quem o acordar deve chamar `sd_compute_snapshot` logo depois — está anotado nos dois lugares.

A lacuna da franja está fechada, e o que ela revelou

O `hms_from_domain` indexa por `mp_lookup(sd_get_domain_mapper(sd), hig_get_cid(c))` e rejeita índice fora de `[0, n)`, onde `n` exclui a franja pela cláusula C6. Essa afirmação só estava exercitada em `np=1`, e com a franja montada à mão pelo próprio teste.

`test-mesh-snapshot-parallel` fecha isso: monta o domínio pelo caminho de produção (`lb_calc_partition`, `psd_create`, `psd_synced_mapper`) e afirma, por redução global, em `np = 1, 2 e 3` e nos dois DIM. **A convenção se sustenta** — os locais ficam em `[0, n)` e a franja vem depois, em todas as configurações.

Mas o exercício revelou algo que muda como se deve ler essa garantia.

A identidade de índice é verdadeira por construção. O `_psd_setmapper` numera os locais com `mp_assign_from_celliterator(m, sd_get_domain_celliterator(sd), 0)` — o *mesmo* iterador que o `hms_from_domain` percorre. O id do mapeador é, por definição, o contador do percurso. Medido: trocar `mp_lookup(m, hig_get_cid(c))` por `n_visitadas++` dentro do `hms_from_domain` deixa a suíte **inteira verde**, nos dois DIM e nos três `np`. Não é oráculo fraco — os dois são a mesma função.

O que resta a proteger, portanto, não é a aritmética do instantâneo: é a **convenção de numeração da produção**. E aí o teste discrimina. Numerando a franja primeiro no `_psd_setmapper`, o teste serial passa nos seus 3 casos (ele imita a convenção à mão) e o paralelo reprova em `np=2` e `np=3`, nos dois DIM.

Isso tem consequência de projeto: o passo 3 da seção 5 — o oráculo diferencial — **não pode** comparar o instantâneo contra o mesmo percurso que o gerou, porque isso é tautologia. Ele tem de alcançar a célula por outra via (busca por ponto) e conferir que o id leva à linha certa.

2. O terreno

laços de célula	(for it = sd_get_domain_celliterator)	233	
sítios de faceta	(sfd_get_domain_facetiterator)	97	
sd_get_domain_celliterator	255	hig_get_center	250
hig_get_cid	207	hig_get_delta	144
sfd_get_stencil	133	sd_get_stencil	67
sd_get_cell_with_point	18		
		TOTAL	1074

O laço canônico — e são 233 variações dele — é exatamente a forma do instantâneo:

```
for(it = sd_get_domain_celliterator(sdp); !higcit_isfinished(it); higcit_nextcell(it)) {
    hig_cell *c = higcit_getcell(it);
    int clid = mp_lookup(m, hig_get_cid(c)); // → i
    Point center; hig_get_center(c, center); // → s→center[i*DIM + d]
    real val = ns→problem→pressure(center, ns→par.t);
    dp_set_value(ns→dpp, clid, val);
}
```

Dos 306 `mp_lookup`, **183 são literalmente o par** `mp_lookup(m, hig_get_cid(c))`, e `psd_get_local_id` não é usado em lugar nenhum. O mapeador é sempre o mesmo objeto que o `hms_from_domain` já consulta (`sd_get_domain_mapper`), então a identidade de índice vale por construção, não por coincidência — uma vez confirmada com franja.

3. Onde o instantâneo vive

Recomendação: no `sim_domain`, **produzido ao fim do** `psd_synced_mapper`. Não no `higflow_solver`, e não na primeira leitura.

Por que no domínio

O argumento é de contagem. O solver declara mais de vinte campos de domínio — `sdp`, `sdF`, `sdED`, `sdmult`, `sdVisc`, `sdSBnA`, `sdSBnB`, `sdphi`, os quatro de eletro-osmose, mais `sfdU[DIM]`, `sfdF[DIM]`, `sfdE0Feo[DIM]`, cada um com seu par `psd` / `psfd`. Pôr um instantâneo ao lado de cada um duplica essa lista, e cada laço migrado passa a ter de escolher o instantâneo certo para o domínio certo. **Emparelhamento errado não dá erro de compilação e dá resultado plausível.** Pendurado no domínio, esse erro deixa de ser possível de cometer.

O ganho colateral é o da fronteira: quem consome deixa de saber qual backend produziu a malha.

Por que ansioso, e não na primeira leitura

Uma versão anterior desta seção propunha `sd_snapshot(sd)` produzindo na primeira chamada. **Está errado**, e pelo mesmo motivo que o teste de franja expôs: o instantâneo depende do mapeador estar atribuído, e quem o

atribui é o `_psd_setmapper`, dentro do `psd_synced_mapper`. Produção preguiçosa deixa essa ordem implícita — chamada antes, ela devolve um instantâneo indexado por lixo, e o sintoma não é falha, são números errados.

A medição mostra que há um ponto único e não ambíguo para produzi-lo:

<code>psd_create</code>	19
<code>psd_synced_mapper</code>	19
<code>mp_assign_from_celliterator</code> em <code>higflow/src</code>	1

Um para um, e o solver não numera domínio nenhum por conta própria — delega. Então o instantâneo nasce onde o mapeador acabou de ficar pronto, e a ordem deixa de ser uma coisa que alguém precise lembrar.

Junto vai o guarda da seção 1: refino depois da montagem tem de falhar alto, e não devolver geometria velha em silêncio.

O custo, e o que mudaria a decisão

O `sim_domain` passa a carregar estado derivado, e é a `HiGTree` que passa a mantê-lo. Isso só é aceitável porque **a malha é imutável depois da montagem**. Se a adaptação em tempo de execução entrar algum dia, esta é a primeira decisão a rever.

A alternativa legítima é manter a `HiGTree` sem memória — uma biblioteca de malha que responde e não lembra. Não é errada; o preço dela são os vinte e poucos campos novos no solver e o emparelhamento manual, que é pagar no lugar onde o erro é silencioso.

4. O que ainda não existe

~~O instantâneo de facetas~~ — **feito**. `hig_facet_snapshot` guarda a **caixa da célula** da faceta, mais `dim` e `dir` por faceta; centro e tamanho saem daí pelas mesmas contas do `hig_get_facet_center` e do `hig_get_facet_delta`.

Guardar a caixa não foi simetria com o caso das células — foi medição. Dos 97 laços de faceta, 95 leem apenas centro, tamanho e id; os outros 2 chamam o buffer de resíduo, que precisa da caixa da **célula**. Fosse só pelos 95, guardar centro e tamanho prontos bastaria.

Produção ansiosa no `psfd_synced_mapper`, oráculo diferencial próprio (`sfd_snapshot_verify`, localizando por `sfd_get_facet_with_point`), e a mesma flag `HIGTREE_VERIFY_SNAPSHOT` o liga.

O `sfd_get_stencil` (133 sítios) continua fora: é topologia, não leitura, e fica atrás do backend como as consultas de ponto.

A caixa da célula no instantâneo — ~~decisão nova~~ feita. O instantâneo guardava centro e delta. Agora guarda a **caixa** (`low` e `high`), e centro e delta saem dela por derivação, com exatamente as mesmas contas do `hig_get_center` e do `hig_get_delta`.

O motivo é que a caixa é o primário: é o que a célula guarda, e os outros dois são derivados dela. Guardando a caixa, toda leitura sai bit a bit igual à de hoje; guardando centro e delta, reconstruir `low = centro - delta/2` é exato em álgebra e **não é exato em ponto flutuante**.

Isso não era preciosismo, e agora está **medido** em vez de afirmado. Os 19 laços cobertos do `hig-flow-io.c` usam `c→lowpoint` e `c→highpoint` como *pontos de interpolação*, e o resultado vai para o VTK que a suíte compara. A perda da reconstrução:

centro - delta/2 ≠ low 1 de 3 células por direção na malha não diádica
~0,6% de 200 mil caixas aleatórias

O caso `reconstruir_o_canto_nao_seria_exato` afirma essa perda, e é o único lugar que guarda a decisão: se alguém "simplificar" o instantâneo de volta para centro e delta, ele é o que acusa.

Uma coisa que eu **não** consegui prender, e vale registrada: a escolha entre formas algebricamente equivalentes do centro. $(lo+hi)/2$, $lo/2+hi/2$ e $lo+(hi-lo)/2$ dão o mesmo bit em 200 mil caixas aleatórias — dividir por dois é exato em binário. Não há ali o que discriminar.

Custo: dois arranjos de $n * DIM$ em vez de dois (mesma memória), e o produtor do t8code passou a preencher a caixa a partir do centroide e do nível.

4b. Produção por rank — feita, em caixas completas

A floresta nasce em `COMM_WORLD`, o t8code já a reparte por curva de preenchimento, e cada rank materializa só o que é dele. Isso tira o gargalo de um processo ter de caber a malha inteira antes de existir partição.

A representação é um conjunto de caixas completas, não uma árvore com buracos. A primeira tentativa usou buracos e não sobrevive: `hig_get_cell_coords_of_point` desreferencia os filhos para ler as caixas deles, e `hig_get_cell_with_point` ainda faz `cell = cell→children[p]` sem testar nulo. O `lbal` também não produz buraco — ele preenche todos os compartimentos, e é por isso que um domínio tem *várias* árvores por rank.

Decompor no nível base é inválido, e isso foi medido. `set_for_coarsening=1` pede ao t8code que não divida família entre ranks, mas a garantia não é recursiva: ela preserva a família do nível mais fino, e uma célula base refinada duas vezes contém família de famílias. Em 3D com $np=2$, **oito** células base saíram divididas — a materialização criava as que faltavam e o domínio ficava com 541 folhas contra as 533 da floresta. Por isso a decomposição desce até o nível em que a posse fecha: célula base completa vira caixa; célula incompleta desce e emite as sub-caixas que forem completas.

Um defeito que só apareceu em 3D: o agrupamento de células base em retângulos conferia apenas uma **linha** da face ao crescer, não a face inteira. Em 2D a linha é a face, então passava; em 3D a caixa engolia células de outro rank.

O caso `caixas_sao_navegaveis` é o que autoriza usar estas árvores como árvore de domínio:

`hig_get_cell_with_point` no centro de cada folha tem de devolver aquela folha. É exatamente o caso que a versão com buracos não passaria.

4c. O exemplo escolhe a fonte da malha — feito

`higflow_set_fonte_de_malha(ns, f, ctx)` instala um gancho que o `higflow_partition_domain` consulta. Sem gancho, a malha vem do arquivo AMR como sempre — e a suíte afirma essa inércia: **33/33** com o gancho presente e não instalado.

O `example2d_Newt` escolhe em tempo de execução: com `HIGFLOW_MALHA=t8code` no ambiente, a malha é produzida por uma floresta do t8code. A unidade do produtor só é compilada quando se passa `T8CODE=<prefixo>` ao `make` do exemplo; sem isso o exemplo não depende da biblioteca.

O problema da referência foi resolvido evitando-o. Gerar uma referência a partir do código recém-escrito só guardaria contra regressão futura. Em vez disso, o `amrs/domain/ch-d-0.amr` descreve uma malha **uniforme** de 160×40 sobre $[0, 8] \times [-1, 1]$, e um cmesh de *brick* do t8code com 160×40 árvores no nível 0 tem exatamente essas células. Mesma malha, produtor diferente → a saída tem de ser **idêntica**:

3 arquivos VTK	byte a byte idênticos
6 campos gravados	byte a byte idênticos

Sem tolerância, sem número gravado.

A fiação foi conferida, porque saída idêntica também é o que se veria se o caminho t8code tivesse caído de volta no AMR: com a fonte sabotada para recusar, a execução termina em erro e não produz VTK nenhum.

A armadilha que custou uma rodada, e já está registrada duas vezes neste repositório: a regra `malha-t8.o`: ficou antes do alvo `ns-example`, e em `make` a **primeira regra explícita é o alvo padrão** — `make` construía só o objeto e retornava 0. Build "bem-sucedido" sem binário.

A restrição real não é tamanho, é cobertura. A suíte de exemplos roda 33 casos; migrar onde ela não chega é mexer sem rede.

1. **Fechar a lacuna da franja** — `test-mesh-snapshot` com `nps=(1,2,3)`. Barato, e é o que torna tudo abaixo confiável.
2. **A produção no `psd_synced_mapper`**, com o guarda contra refino pós-montagem. Nenhum sítio migrado ainda; a suíte inteira tem de continuar idêntica.
3. **O oráculo diferencial** — `sd_snapshot_verify` localiza cada célula **por ponto** e exige que ela volte com o mesmo id e a mesma geometria. Com `HIGTREE_VERIFY_SNAPSHOT` no ambiente, todo domínio que o programa montar passa por ele, e divergência aborta. Os 33 casos da suíte de exemplos passam com o modo ligado.

Ele **não** percorre o iterador, e isso não é detalhe: o instantâneo foi preenchido pelo iterador indexado pelo mapeador, e o mapeador saiu desse mesmo iterador. Conferir um contra o outro mede a aritmética contra ela própria. 4. **Migrar os laços cobertos por exemplo**, um arquivo por vez, com a suíte entre cada um. 5. **Os solvers sem exemplo, por último** — ou não. Ver a decisão D3.

O caminho de árvore **não sai**. O instantâneo é aditivo: um laço migrado lê do arranjo, os demais seguem lendo da árvore, e os dois descrevem a mesma malha enquanto o oráculo do item 3 passar.

4d. O produtor por rank alimenta o exemplo

`HIGFLOW_MALHA=t8code-rank`: a floresta nasce em `COMM_WORLD`, o t8code a reparte, e **cada rank materializa só a sua parte**. Nenhum processo chega a ter a malha inteira.

O bloqueio que eu tinha previsto não precisou ser resolvido. Eu havia escrito que isso exigiria produzir o `partition_graph` sem o `lbal` — e o `pg` guarda o pareamento de franjas em ordem idêntica nos dois lados, que é o coração do `build-fringe`. Mas o `lbal` **já** espera entrada distribuída: no caminho AMR cada rank lê um subconjunto dos arquivos. Basta cada rank contribuir as suas caixas, e o grafo de vizinhança e a franja saem prontos.

A partição final continua sendo a do `lbal`, não a do `t8code`. O que muda é de onde vem a malha e quanto cada processo precisa segurar.

O oráculo muda com o np, e descobrir isso foi metade do trabalho.

```
np=1  byte a byte idêntico ao caminho AMR  (17 arquivos)
np>1  NÃO é idêntico — e nem podia ser
```

Comparar o caminho AMR **consigo mesmo** em `np=1` contra `np=2` já dá 3 de 6 campos diferentes: o condicionador `bjacobi` depende da partição, então a partição sozinha move os números. Byte a byte não é oráculo disponível em `np>1`, para fonte nenhuma. Ali vale a referência gravada, com tolerância — a mesma que a suíte usa:

```
np = 1, 2, 3 · AMR, t8code, t8code-rank · todas PASS a 0,10%
```

O gancho passou a devolver várias árvores (`int f(ctx, hig_cell **v, int max)`), porque a região de um rank não é uma caixa e vira um conjunto delas.

4e. A partição final pelo t8code — feita

Sem `lb_calc_partition`: cada rank fica com o que a curva de preenchimento lhe deu, e o `partition_graph` é montado a partir das caixas (`t8_monta_dominio_particionado`).

O que tornou isso possível foi a medição: a região de um rank é quase um retângulo — `np=2 → 1` caixa, `np=3 → 2` a 3, `np=4 → 1`, `np=8 → 1` — e o `pg` representa o que se envia como faixa **retangular** em coordenadas de árvore.

O contrato, lido do `pdomain.c` e não suposto: quem envia enumera os elementos de `local_tree` no intervalo `[lo_idx, hi_idx)`; quem recebe enumera a árvore de franja **inteira**. Logo a *i*-ésima faixa enviada tem de ter a mesma forma e a mesma ordem da *i*-ésima árvore recebida.

A regra de ordem é a peça delicada: em ambas as listas, a caixa **dona do dado** fica no laço de fora e a que **precisa** dele no de dentro. Os dois lados calculam o mesmo objeto a partir dos mesmos dados globais (um `allgather` das caixas), então nada é negociado por mensagem.

Eu errei isso na primeira versão — o envio aninhava (*minhas, dele*) e o recebimento também, mas o remetente emite com as caixas *dele* por fora, então as listas saíam **transpostas**.

```
transposição do aninhamento
np=2  passa  (uma caixa por rank: 1×1 esconde)
np=3  REPROVA 228 de 304 células de franja com valor de outra posição
```

Um teste só em `np=2` não pegaria esse defeito — e nenhum teste de *contagem* ou *volume* pegaria em `np` nenhum.

Limite declarado: malha uniforme. Com refino, a faixa do remetente e a árvore do receptor precisariam ter a mesma subdivisão interna, e isso exige trocar estrutura, não só índices.

4f. Malha e partição do t8code, no exemplo

`HIGFLOW_MALHA=t8code-particao` : o solver não chama `lb_calc_partition` . A malha e a partição são do t8code de ponta a ponta.

O gancho novo, `higflow_set_fonte_de_particao` , difere do de malha: aquele entrega **árvores** para o `lbal` reparticionar; este entrega o resultado **já particionado** — domínio com franja e grafo de vizinhança.

```
np = 1, 2, 3 · referência a 0,10% · PASS
```

Falhar aqui aborta, e é deliberado: seguir com domínio de vizinhança errada dá resultado plausível e errado. Conferido por sabotagem — com a fonte recusando, a execução não gera VTK nenhum.

Uma coisa que a integração forçou e melhorou o código: a cadeia de doze domínios que recebem cada árvore estava embutida no laço do `lbal` . Virou função (`_adiciona_aos_dominios`), porque o caminho do t8code precisa da mesma cadeia — e duplicar um `if` -chain de doze domínios é como os campos do `ns→cc` se perderam.

As quatro fontes, e o que cada uma exercita:

(omissão)	AMR de arquivo, <code>lbal</code> reparticiona
<code>t8code</code>	malha do t8code em série, <code>lbal</code> reparticiona
<code>t8code-rank</code>	malha por rank, <code>lbal</code> reparticiona
<code>t8code-particao</code>	malha e partição do t8code, sem <code>lbal</code>

4g. A suíte que roda nos dois backends

`python3 ci/run_suite.py --t8code <prefixo>` : além do caminho AMR, cada caso roda as fontes que declara em `t8_fontes` , **contra a mesma referência**.

sem a flag	33 de 33	(só o caminho AMR — nada muda)
com a flag	75 de 75	(AMR + variantes t8code)

Em 21/09 entrou o primeiro caso 3D (`example3d_complex` , 33 blocos, 162 mil células), e com ele o limite que só a tentativa revela: **das três fontes, só `malha_t8_uniforme` tem verificação 3D**. As outras duas recusam `numhigs ≠ 1` — cobrem um bloco só — e não existe exemplo 3D de bloco único na suíte. A recusa é por `abort` , de propósito:

```
particao_t8: 33 blocos; esta fonte cobre um so'  
higflow_partition_domain: a fonte de particao falhou. Seguir daqui daria  
dominio com vizinhanca errada, que produz resultado plausivel e errado.
```

Vale registrar o que aconteceu ali, porque é o oposto de tudo o mais neste documento: **o código recusou parecer verde**. Eu havia listado a fonte de partição por analogia com os casos 2D, sem conferir o limite dela; a guarda me

corrigiu em vez de me entregar números plausíveis.

A fonte deixou de ser decorada. Ela lê a especificação do próprio arquivo AMR (`mi→l` , `mi→h` , `levels[0].patches[0].patchsize`), então serve a qualquer exemplo de malha uniforme. Com refino ela **recusa** em vez de aproximar — malha diferente dá números diferentes, e o silêncio ali seria pior que o erro.

O levantamento decidiu a elegibilidade, e vale registrado:

8 exemplos	um bloco uniforme
1 exemplo	DOIS blocos uniformes (<code>Newt_contraction</code>)
5 exemplos	sem arquivo de domínio (constroem a malha por outro caminho)

Sete estão ligados. O `BMP` ficou de fora: ele inicializa o domínio por outro caminho, e meia ligação é pior que nenhuma.

O custo por exemplo é **uma linha** (`malha_t8_instala`) mais o bloco `T8CODE` no Makefile — o instalador compartilhado evita repetir o `strcmp` de quatro fontes em sete lugares, que é como os campos do `ns→cc` se perderam.

Dois defeitos que isto expôs:

A fonte em série fazia **todo rank produzir a malha inteira**, em vez de distribuir os blocos como o caminho AMR faz (`for i = myrank; i += ntasks`). Ficou latente porque ela só tinha sido exercitada em `np=1`, onde a regra é trivialmente satisfeita; em `np=2` a referência reprovou.

E o meu próprio contador: ao passar a contar execuções reais para incluir as variantes, os `skip` saíram do total e "33/33" virou "31/31" sem que nada tivesse regredido. O número de cabeçalho da suíte é comparado entre execuções — mudá-lo em silêncio é um defeito, não um detalhe.

6. O que é decisão sua

D1 — Onde o instantâneo vive, e quando nasce — decidido, como recomendado. No domínio, produzido ao fim do `psd_synced_mapper` . A alternativa era no solver, que manteria o `sim_domain` sem estado derivado ao preço de vinte e poucos campos novos e do emparelhamento manual.

D2 — Os cinco solvers que nenhum exemplo alcança — decidido CONTRA a minha recomendação: migrar mesmo assim. Eu recomendei não migrar sem exemplo que os exercite, e o usuário decidiu migrar; estão migrados.

Registro o que isso significa, porque a decisão não muda o fato: **aqueles sítios compilam e não são exercitados por nada**. Compilar e ser exercitado são coisas diferentes, e só a primeira está estabelecida ali. O mesmo vale, por outro motivo, para o `high-flow-res.c` : ele só entra no build com `make RES=1` , que nenhuma suíte usa. Verifiquei que ele compila nas duas dimensões com a assinatura nova do buffer de resíduo — pelo comando real extraído de `make -n -B RES=1` , e sem acrescentar um aviso sequer — mas ninguém nunca **rodou** aquele caminho, então os números que ele produz não foram conferidos contra nada.

D3 — A classificação na interface entre árvores — decidido. Vale a semântica do `t8code`: *sobre o contorno* significa que o domínio **termina** ali. Interface entre blocos do mesmo domínio não é contorno, porque há malha dos dois lados.

A investigação mudou o caráter da decisão. Não eram duas convenções defensáveis — o `cell_find_in_center` para no **primeiro** higtrees cuja caixa contém o ponto (`break` , "assume domains do not overlap") e nunca pergunta se outro bloco continua o domínio. Isso faz a resposta depender da **ordem das árvores**, que é o que a cláusula C8 proíbe para localização.

E não é hipotético: medido, com `np=3` um domínio do `example2d_Newt` chega a **três** higtrees, então as interfaces internas existem em toda execução paralela.

O efeito hoje é **benigno**, e rastreei até o fim: `ON_BOUNDARY` faz o `get_stencil` procurar condição de contorno — Dirichlet no centro, Neumann, Dirichlet — e numa interface interna nenhuma casa, então cai no caminho normal de interpolação. O preço é busca desperdiçada na camada de facetas daquele plano.

O que não fica garantido é que nenhuma condição registrada coincida com uma interface interna; se coincidir, seria aplicada num ponto interno. Corrigir exige perguntar se o domínio *continua* além do ponto — uma sonda com epsilon — e isso merece verificação própria em vez de vir de carona nesta decisão. Fica escrito na C14 como desvio conhecido do MTree, não como convenção alternativa.

7. Onde isto parou — 21 de setembro de 2026

A arquitetura que resultou

Duas costuras novas, e nenhuma delas obriga quem não as usa.

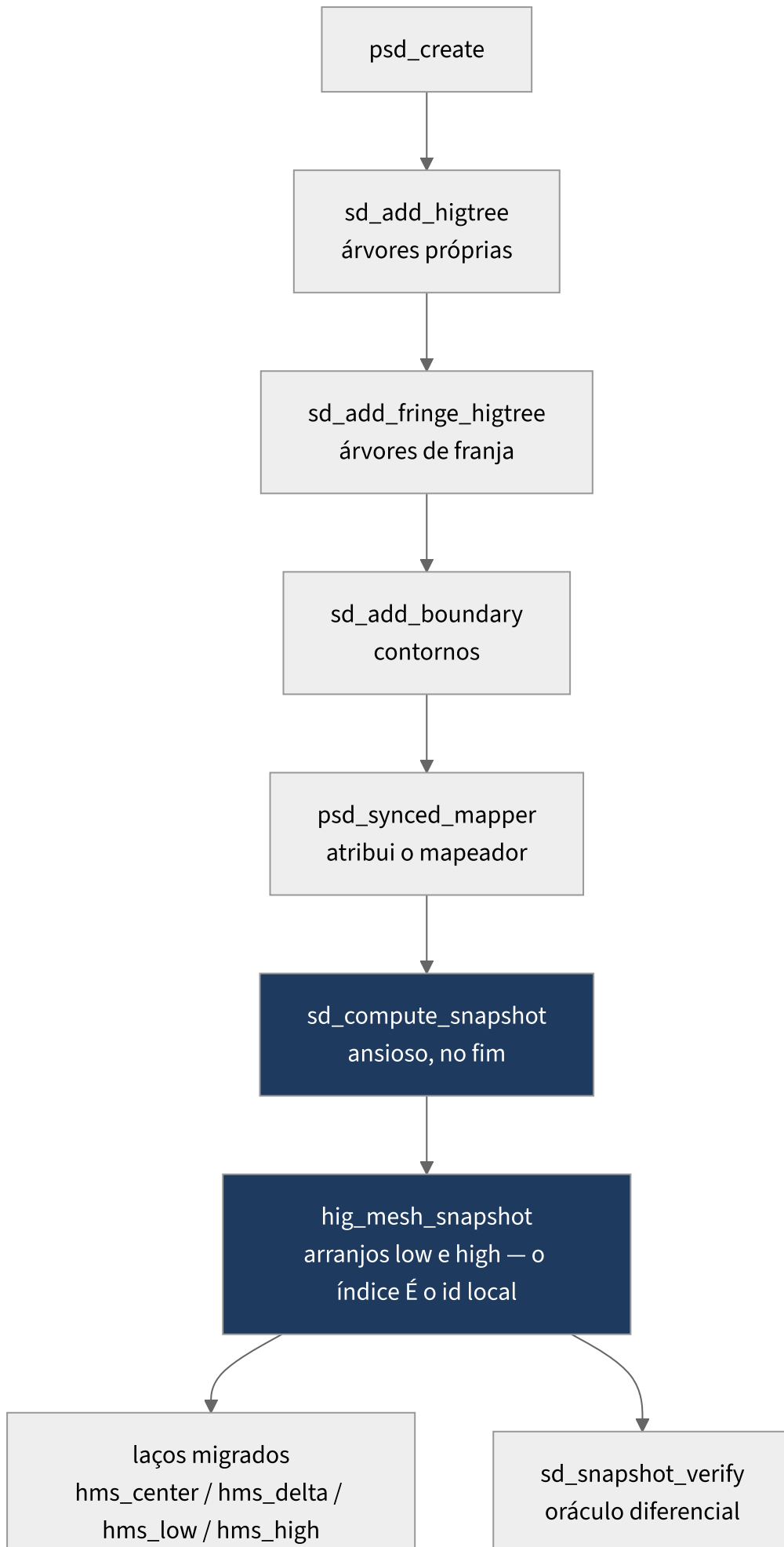
Na HiGTree, a fronteira das consultas mudou de lugar. O `hig_mesh_snapshot` e o `hig_facet_snapshot` são arranjos planos em que **o índice é o id local**, pendurados no `sim_domain` e no `sim_facet_domain`, produzidos ao fim do `psd_synced_mapper` / `psfd_synced_mapper`. Guardam a **caixa**, não centro e tamanho, porque a caixa é o primário e reconstruí-la não é exato em ponto flutuante. Quem lê geometria não precisa mais de um `hig_cell`, e é isso que permite uma segunda implementação de malha responder sem materializar árvore de ponteiros.

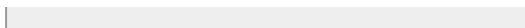
No HiGFlow, o exemplo escolhe a fonte da malha em uma linha. Dois ganchos no `higflow_solver` (`fonte_de_malha`, `fonte_de_particao`), zerados na criação, e o `higflow_partition_domain` com três caminhos. Sem `HIGFLOW_MALHA` no ambiente, nada muda — o caminho legado não foi removido nem contornado, e a suíte padrão afirma isso em cada execução.

O contrato de Mesh (`higtrees/src/hig-mesh-contract.h`, 18 cláusulas) é o que torna as duas implementações comparáveis: ele é lido do próprio cabeçalho pelo driver, então cláusula sem caso de teste não passa despercebida.

HiGTree — a ordem de construção é parte do contrato

O estado derivado nasce uma vez e passa a ser confiado. Por isso a ordem não é convenção: `sd_add_higtrees` **recusa** rodar depois que o instantâneo existe.





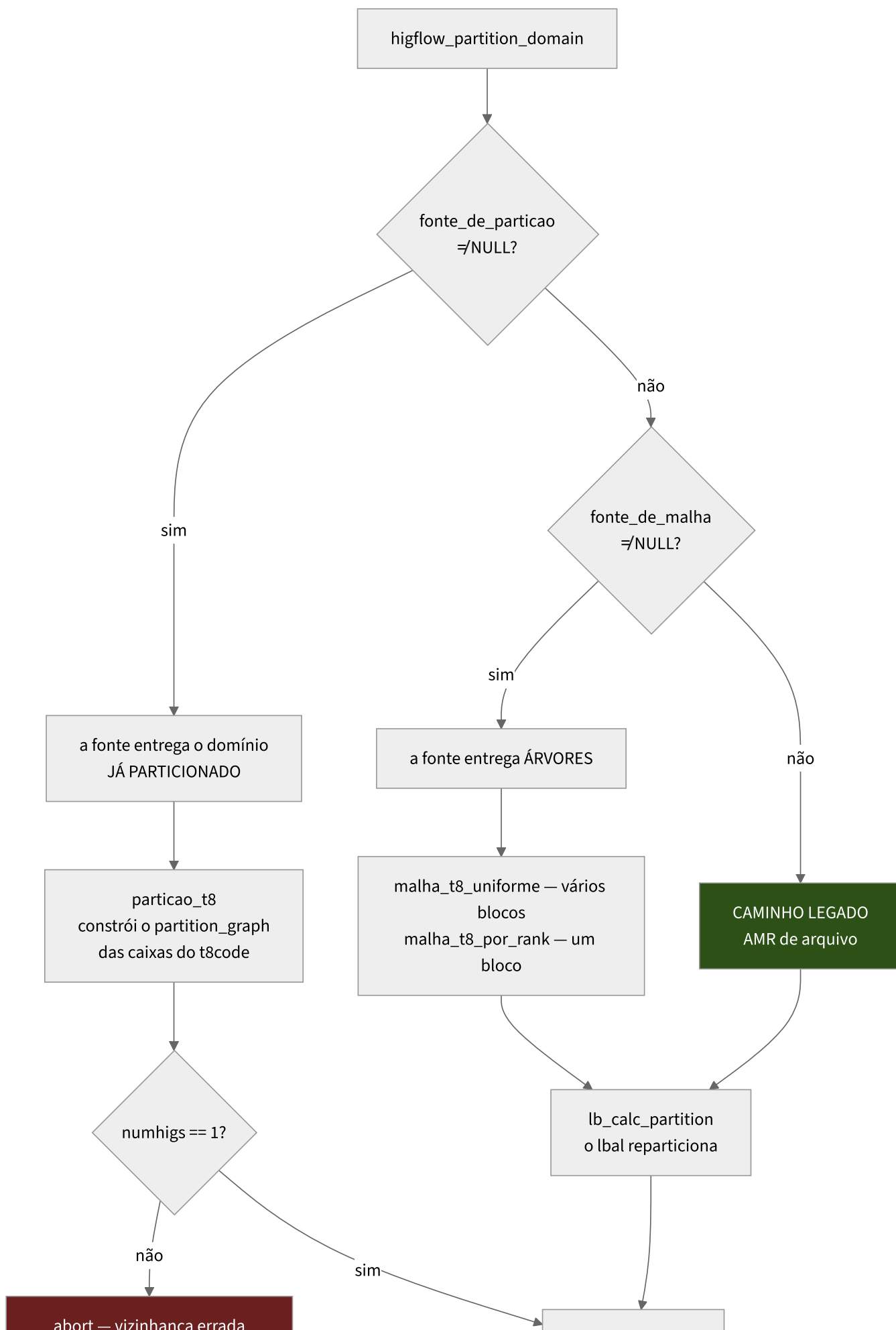
localiza por PONTO,
não pelo iterador



diverge → abort

O oráculo localiza por ponto porque o mapeador é atribuído a partir do **mesmo** iterador que o instantâneo percorre: compará-los mediria a aritmética contra si mesma.

HiGFlow — os três caminhos, e o legado intacto no meio

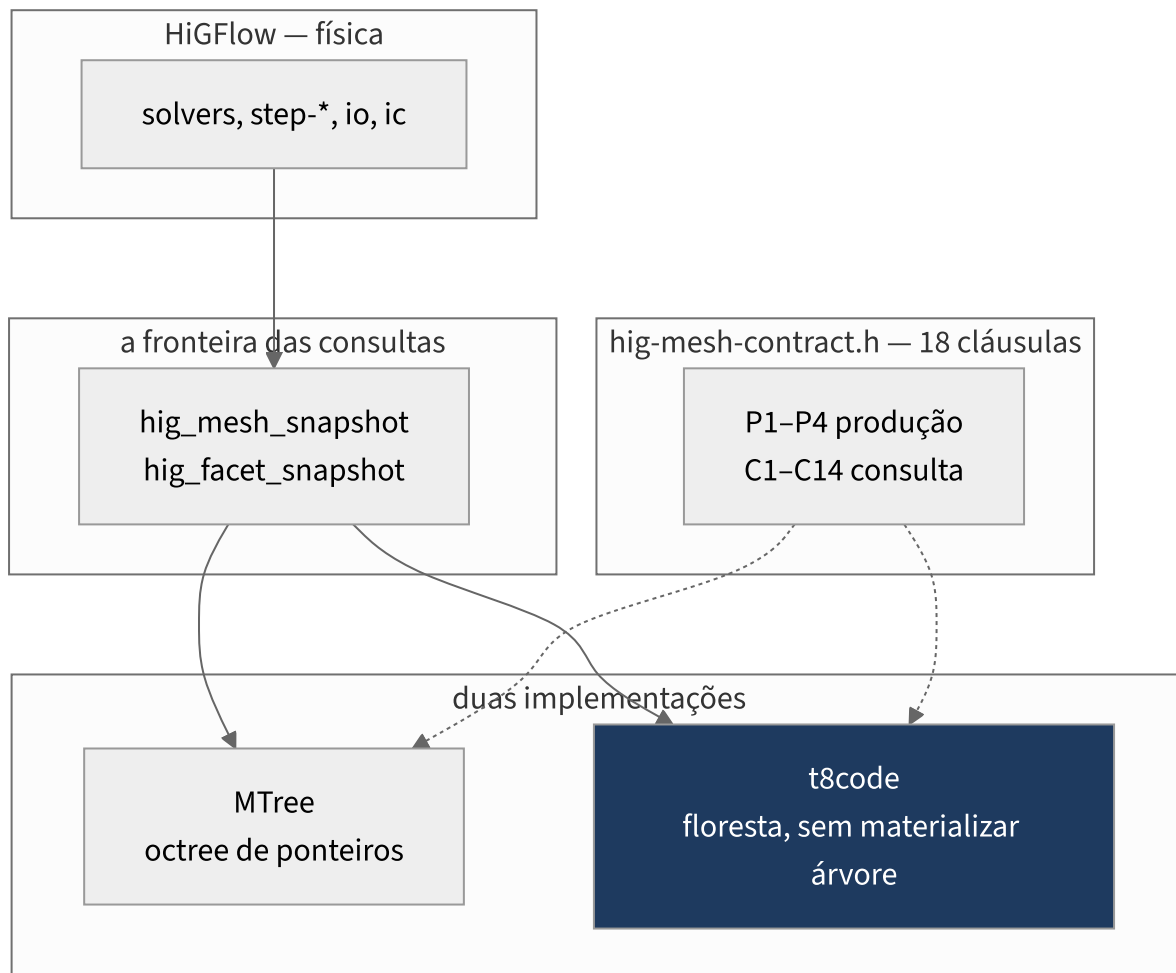


daria número plausível e
errado

domínio pronto
SEM lbal

Sem `HIGFLOW_MALHA` no ambiente, os dois ganchos são `NULL` e a execução desce pelo caminho verde — o legado não foi removido nem contornado, e a suíte padrão afirma isso a cada execução.

As duas implementações de malha, e o que as compara



O driver lê as 18 cláusulas **do próprio cabeçalho** e as confronta com os casos existentes, então cláusula sem caso não passa despercebida — é o que impede a suíte de crescer afirmando menos do que declara.

O `t8_preenche_instantaneo` preenche o arranjo plano **direto da floresta**, sem construir árvore de ponteiros alguma. É esse o ponto da costura: o custo que se queria evitar era a tradução, e guardar a caixa em vez de centro e tamanho é o que a torna desnecessária.

O que está verificado, e por qual rede

HiGTree	316 de 316 casos, contrato de Mesh 18 de 18
HiGFlow	75 de 75 execuções (7 exemplos × fontes, contra a MESMA referência)
oráculo	HIGTREE_VERIFY_SNAPSHOT=1 confere todo domínio por LOCALIZAÇÃO DE PONTO, e aborta na divergência

O oráculo localiza por ponto de propósito: o mapeador é atribuído a partir do **mesmo** iterador que o instantâneo percorre, então conferir um contra o outro mediria a aritmética contra si mesma.

O que continua em aberto, e por que não foi fechado

As duas funções de refino no lugar, e o teto de 40. Estas duas coisas estavam em seções separadas deste documento e são, na verdade, a mesma:

O `sim_facet_domain.sfbi[]` é arranjo **fixo** de 40 (`MAXHIGTREESPERDOMAIN`) e o `sfd_set_sfbi` o indexa **sem conferir limite**, enquanto o arranjo de árvores do domínio cresce sem teto. Medido no pior caso do repositório:

```
example3d_complex    np=1    33 = 33 próprias + 0 de franja    (índice máximo 32)
                     np=2    23 = 20 + 3
                     np=3    18 = 13 + 5
                     np=4    17 = 10 + 7
                     np=6    16 = 10 + 6
```

O crescimento vem das **próprias**, não da franja, e o máximo cai monotonicamente: mais ranks é mais seguro. O pior caso é `np=1`, com sete de folga.

A ligação é esta: **o número só pode crescer sem passar por partição nova se alguém acordar**

`higflow_refine_tree_inplace` ou `adapt_mesh_and_update_solver`. O caminho de adaptação alcançável hoje (`higflow_rebuild_with_amr`) reconstrói e reparticiona, então respeita a medição acima. As duas funções dormentes não passam pela API do domínio — e são, ao mesmo tempo, o que deixaria o instantâneo velho em silêncio e o que poderia estourar o teto de 40. Quem as acordar precisa chamar `sd_compute_snapshot` e reexaminar o teto.

As fontes de rank e de partição não têm verificação 3D, e não por falta de tentativa: cobrem um bloco só, e não existe exemplo 3D de bloco único na suíte.

O `sfd_get_stencil` (**133 sítios**) segue no **caminho antigo**. É topologia, não leitura de geometria, e fica atrás do backend junto com as consultas de ponto.

Como reproduzir tudo isto do zero

```
./instalar.sh --verificar
```

Constrói o t8code do fonte, as duas bibliotecas, os exemplos, e roda as duas suítes. Testado em árvore limpa. O tutorial de uso das duas malhas está em `doc/tutorial-mtree-t8code.pdf`.